

SPRAWOZDANIE

Techinka cyfrowa
Ćwiczenie 4. Wąż FPGA.

Fertig Olaf
Małek Marek
Mesek Robert
Sajacki Bartosz

30 kwietnia 2026

1 Wstęp do układów FPGA

Układy FPGA (Field-Programmable Gate Arrays) są rodzajem układów scalonych, które pozwalają modyfikować ich działanie poprzez opisywanie architektury płytki. To oznacza, że zamiast być zaprogramowanymi w sposób stały podczas produkcji, jak tradycyjne układy scalone, mogą być konfigurowane lub “programowane” przez użytkownika po ich zakupie. FPGA składają się z wielu bloków logicznych, które mogą być programowane do wykonywania różnych funkcji logicznych. Te bloki logiczne są połączone w sposób, który pozwala na tworzenie złożonych ścieżek sygnałowych, umożliwiając realizację różnorodnych zadań. Programowanie FPGA polega na definiowaniu połączeń i konfiguracji wewnętrznych bloków logicznych.

2 Przykładowe wykorzystania układów FPGA

Układy FPGA dzięki prostocie użycia i jednocześnie możliwości zaprojektowania działania pod konkretny problem mają bardzo szeroką listę zastosowań. W telekomunikacji pełnią kluczową rolę w urządzeniach do przetwarzania sygnałów, takich jak routery czy bramy, gdzie zapewniają szybkie przetwarzanie danych i filtrowanie sygnałów. W branży wojskowej i lotniczej wykorzystywane są w systemach kontroli lotu, radarach oraz w kryptografii. W dziedzinie systemów wbudowanych są niezbędne dla urządzeń medycznych, samochodowych systemów kontroli oraz urządzeń IoT, umożliwiając szybkie dostosowanie się do zmieniających się wymagań aplikacji. Ponadto, FPGA są używane w cyfrowym przetwarzaniu sygnałów, umożliwiając implementację filtrów, algorytmów kompresji danych oraz przetwarzania obrazu. Dzięki swojej elastyczności i możliwości szybkiego prototypowania, układy FPGA są niezwykle wartościowe, wspierając twórców oprogramowania w szybkim wdrażaniu nowych rozwiązań.

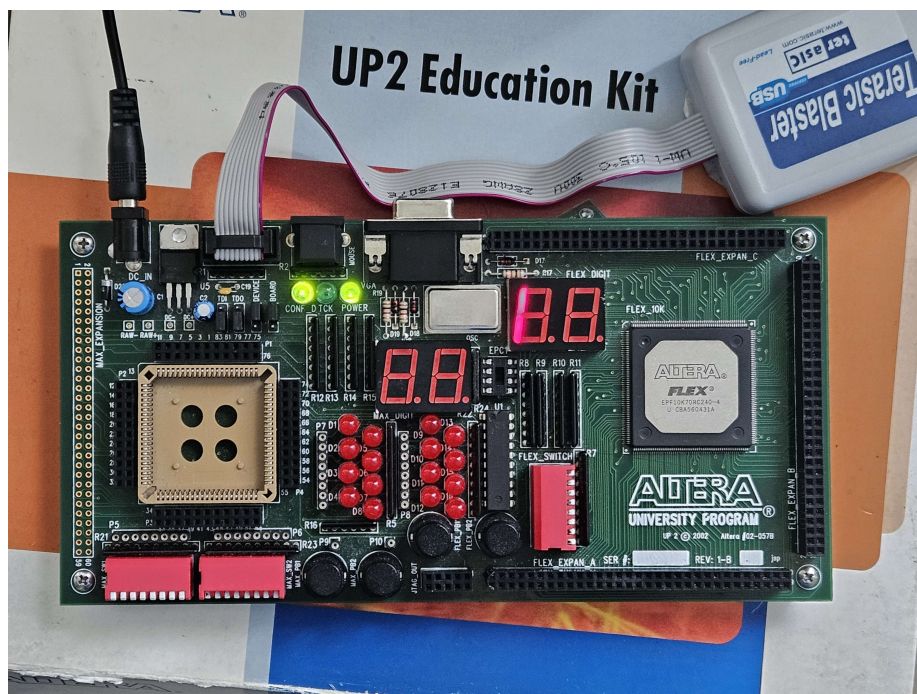
3 Opis problemu

Korzystając z programu Quartus firmy Intel (www.intel.com), należy zrealizować na dowolnym układzie FPGA (np. w zestawach: UP2 (rys. 1), DE2 lub DE2-70 dostępnych w laboratorium) projekt efektu świetlnego wyświetlanego na dwóch wyświetlaczach siedmiosegmentowych, polegającego na przemieszczaniu się po obrzeżach tych wyświetlaczy w określonym kierunku odcinka świetlnego składającego się z dwóch segmentów, zgodnie z filmem z poniższego linku.

<http://home.agh.edu.pl/~dlugopol/tc24/fpga-efekt.mp4>

Po chwilowym krótkim wciśnięciu pierwszego z dwóch przycisków (lewego), układ powinien zmienić prędkość poruszania się odcinka świetlnego.

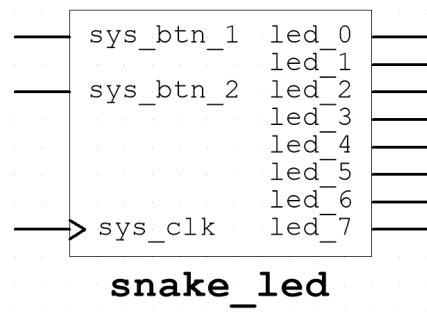
Po chwilowym krótkim wciśnięciu drugiego z dwóch przycisków (prawego), układ powinien zmienić kierunek poruszania się odcinka.



Rysunek 1: Płytki FPGA - Altera UP2

4 Propozycja rozwiązania

Projekt zdecydowaliśmy się napisać w języku SystemVerilog. Pomocne okazało się zaobserwowanie, że podświetlanie poszczególnych segmentów dwóch wyświetlaczy siedmiosegmentowych można zamodelować jako stan cyklicznego rejestru przesuwającego o długości **8** (tyle jest segmentów, które będą używane), w którym inicjalnie znajduje się pewna liczba bitów **0** oraz **1**. W naszym przypadku **0** będzie oznaczało wyłączony segment, a **1** podświetlony segment. Rozpoczęliśmy od stanu wynoszącego “**11000000**”, ponieważ wtedy najlepiej widoczny był efekt poruszania się “węża”.



Rysunek 2: Główny moduł projektu - `snake_led`

Napisany został kod opisujący poszczególne moduły:

- Główny moduł projektu - `snake_led.v`
- Synchronizator - `synchronizer.v`
- Licznik czasu - `timer_parameter.v`
- Automat obsługujący eliminację drgań - `debouncer_delayed_fsm.v`
- Moduł eliminujący drgania wraz z timerem - `debouncer_delayed.v`
- Detektor krawędzi - `edge_detector.v`
- Moduł przycisku - `button.v`
- Przerzutnik typu T - `t_ff.v`
- Cykliczny rejestr przesuwający z wyborem kierunku - `shift_reg.v`

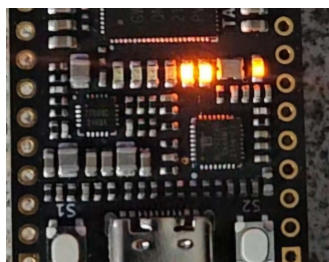
następnie wykorzystano te komponenty do stworzenia projektu pełzającego węża (Rysunek 2). Każdy z tych modułów został zaimplementowany w osobnym pliku, co poprawiło przejrzystość kodu oraz podział całego projektu na poszczególne moduły.

4.1 Główny moduł projektu

Całość projektu łącząca podmoduły, realizująca cel zadania. Składa się on z: dwóch modułów filtrujących przycisków służących do zmieniania stanu przerzutników przechowujących kierunku oraz prędkości poruszania się naszego “węża”; dwóch przerzutników typu T utrzymujących stan kierunku i prędkości; cyklicznego rejestru przesuwającego przechowującego aktualną pozycję “węża” oraz szybkiego i wolnego zegara obsługujących przesuwanie rejestru. Wejściami są systemowe sygnały zegara oraz dwóch przycisków. Wyjście jest rejestrem o konfigurowalnej długości. Ciekawą obserwacją jest fakt, że “wąż” poruszający się po wyświetlaczach siedmiosegmentowych, a diodach LED ustawionych obok siebie to przypadek różniący się jedynie kombinacją podpięcia sygnałów (Rysunki 3 i 4).

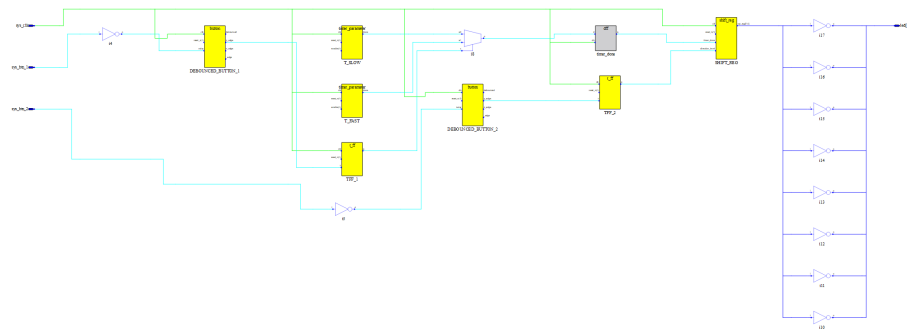


Rysunek 3: Dwa wyświetlacze siedmiosegmentowe



Rysunek 4: Rząd sześciu diod LED

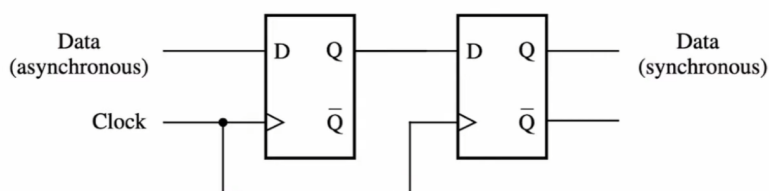
Kod implementacji został przedstawiony w sekcji 8.1. Mamy już więc moduł z rysunku 2, pozostaje nam podpiąć odpowiednie wejścia oraz wyjścia przed załadowaniem programu na konkretną płytkę. Schemat wygenerowany przez język SystemVerilog przedstawiony jest na rysunku 5.



Rysunek 5: Schemat modułu snake_led

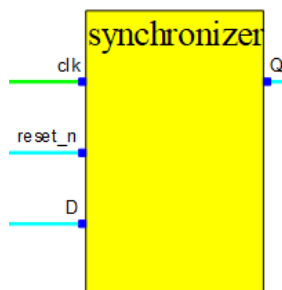
4.2 Synchronizator

Synchronizatory służą do dopasowania sygnałów pochodzących z różnych źródeł, które działają z różnymi częstotliwościami zegara. Jego zadaniem jest zapobieganie problemom związanym z niestabilnością sygnału (tzw. metastabilnością), które mogą wystąpić, gdy sygnał próbkowany jest przez układ zegarowy niezgodny z jego własnym zegarem. Synchronizator zapewnia, że sygnały wejściowe zostaną odpowiednio przetworzone.

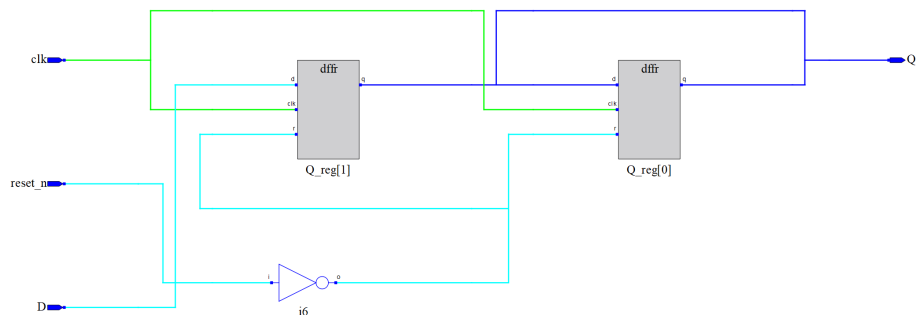


Rysunek 6: Schemat synchronizatora wykorzystującego 2 przerzutniki

Okazał się on kluczowy, przy projektowaniu modułu przycisku. Kod implementacji został przedstawiony w sekcji 8.2. Blok podukładu przedstawiony jest na rysunku 7. Implementacja w postaci schematu widoczna jest na rysunku 8.



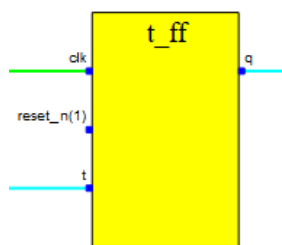
Rysunek 7: Blok podukład synchronizatora



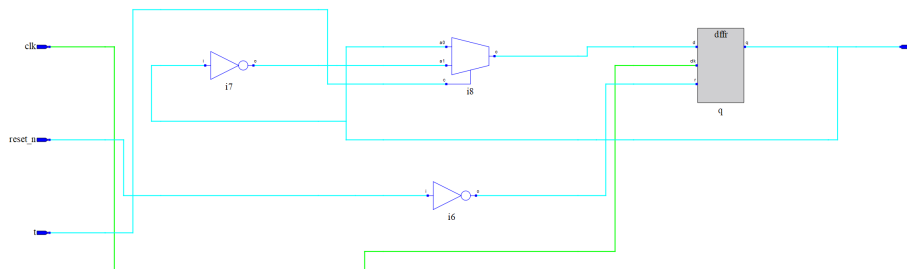
Rysunek 8: Schemat podukładu synchronizatora

4.3 Przerzutnik typu T

Przerzutnik typu T został zaimplementowany samodzielnie, jako jeden z pierwszych komponentów, ponieważ pozwolił on na prostym przykładzie zrozumieć sposób działania opisu sprzętu języka SystemVerilog. Sterując nim przyciskami, wykorzystaliśmy go, jako selektor prędkości oraz kierunku przesuwania “węża”. Działanie przerzutnika jest uzależnione od zegara (wejście `clk`), jest więc to przerzutnik synchroniczny, ale posiada także aktywne zerem asynchroniczne resetowanie do stanu początkowego (wejście `reset_n`). Kod implementacji został przedstawiony w sekcji 8.8. Blok podukładu przedstawiony jest na rysunku 9. Implementacja w postaci schematu widoczna jest na rysunku 10.



Rysunek 9: Blok podukładu przerzutnika typu T

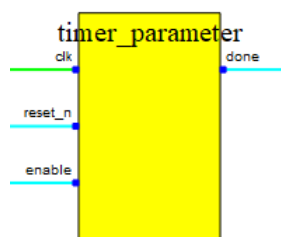


Rysunek 10: Schemat podukładu przerzutnika typu T

4.4 Miernik czasu

Nasze rozwiązanie drgań przycisku będzie opierało się na potrzebie przytrzymania przycisku w danym stanie przez pewien okres czasu ignorując krótkie sygnały spowodowane drganiem styków przycisku. Swoje zastosowanie ten licznik znajdzie także podczas obracania rejestru (“poruszania węzłem”), ponieważ nie chcemy aby poruszał on się z prędkością zegara systemowego (na udostępnionej płytce FPGA około 25MHz). Będzie w pewnym sensie dzielnikiem zegara systemowego, dający sygnał jedynie po określonej parametrem ilości cykli zegara.

W praktyce jest to licznik, który po dojściu do określonej liczby daje sygnał **done** następnie wraca do 0 i ponawia odliczanie. Ma on także dodatkowe wejścia pozwalające na zatrzymanie oraz resetowanie licznika. Kod implementacji został przedstawiony w sekcji 8.3. Blok podukładu przedstawiony jest na rysunku 11.



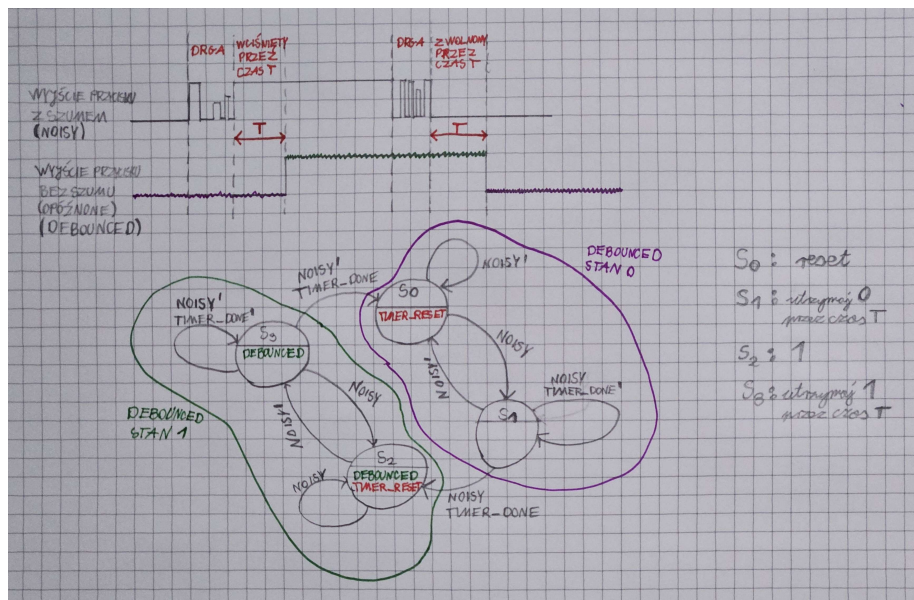
Rysunek 11: Blok podukładu miernika czasu

4.5 Automat filtrujący drgania

Głównym problemem do rozwiązania w tym projekcie okazała się obsługa przycisków. Konceptyjnie przycisk jest bardzo prosty, posiada dwa stany “rozłączony” oraz “załączony”, ale w rzeczywistości jest bardziej skomplikowany. Pod-

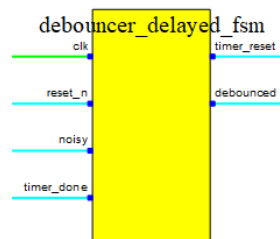
czas wciskania przycisku jego styki zanim osiągną i utrzymają się w danym stanie wpadają w drgania. Nawet wiele milionów razy na sekundę mogą się stykać i rozwierać. Potrzebujemy więc sposobu na odfiltrowanie powstałego szumu. Można to osiągnąć na kilka sposobów m.in. bardziej elektryczne rozwiązanie z zastosowaniem kondensatora lub wykorzystując rejestr do którego synchronicznie ładujemy próbki sygnału, czekając aż będzie wypełniony samymi 1.

Tutaj natomiast zastosujemy automat skończony Moore'a, który w połączeniu z timerem, będzie oczekiwać, aby przycisk pozostał w określonej pozycji przez pewien okres. Zaprojektowany automat został przedstawiony na rysunku 12.



Rysunek 12: Automat skończony Moore'a filtrujący drgania

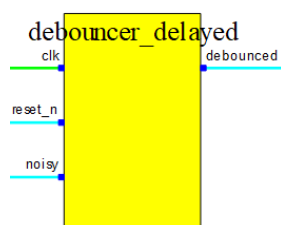
Kod implementacji został przedstawiony w sekcji 8.4. Blok podukładu przedstawiony jest na rysunku 13.



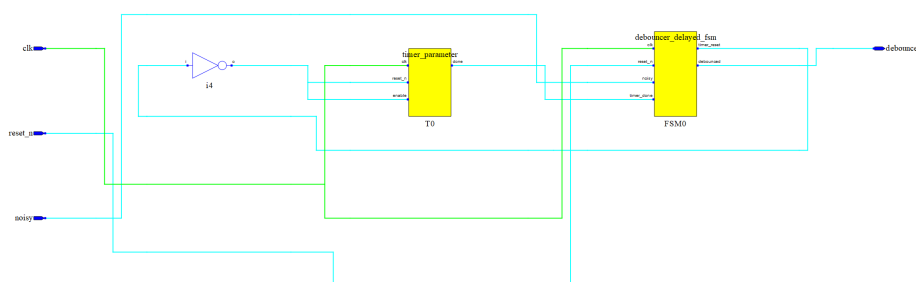
Rysunek 13: Blok podukładu automatu filtrującego drgania

4.6 Filtr drgań (opóźniony)

Pozostaje połączyć moduł automatu filtrującego drgania wraz z modułem miernika czasu z odpowiednio dobranym parametrem limitu. W naszym przypadku metodą prób i błędów liczbą jednocześnie pozwalającą na szybkie klikanie przycisku oraz rejestrowanie jedynie pojedynczych kliknięć okazała się liczba około 2 milionów cykli zegara systemowego. Kod implementacji został przedstawiony w sekcji 8.5. Blok podukładu przedstawiony jest na rysunku 14. Implementacja w postaci schematu widoczna jest na rysunku 15.



Rysunek 14: Blok podukładu filtru drgań

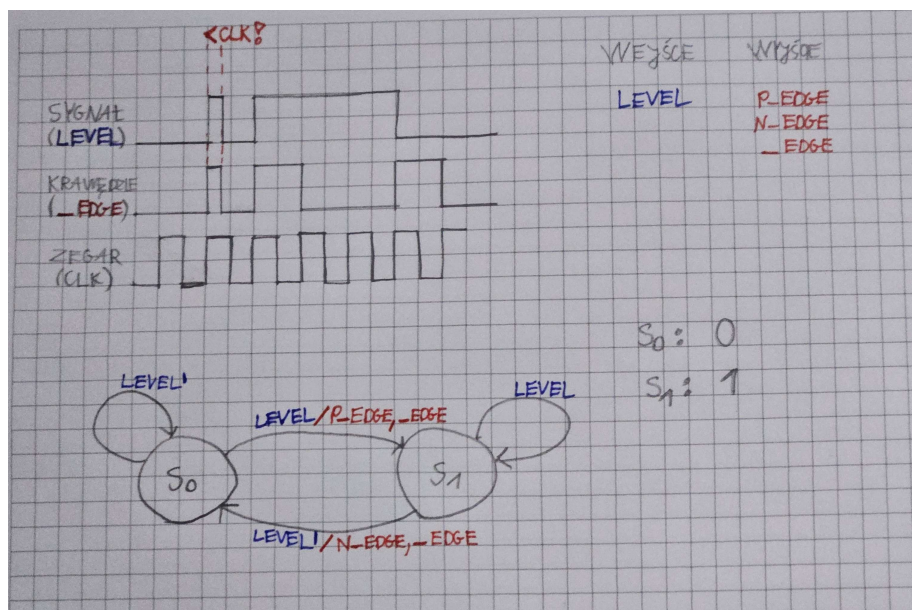


Rysunek 15: Schemat podukładu filtru drgań

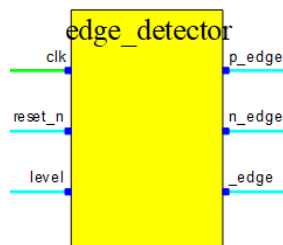
4.7 Detektor krawędzi

Mamy już odsumiony sygnał przycisku, ale na potrzeby naszego układu potrzebujemy otrzymać informację o wciśnięciu przycisku nie ciągły sygnał od niego (w takim przypadku zmiany na przełącznikach typu T powtarzały by się z prędkością zegara systemowego przez cały okres kiedy przycisk jest wciśnięty). Zaprojektujemy, więc automat Meal'ego. Warto zwrócić uwagę, że sygnał wykrycia krawędzi może trwać krócej niż cykl zegara systemowego, ale dokładnie w tym celu możemy wykorzystać zaprojektowany wcześniej synchronizator. Ta sytuacja oraz automat został przedstawiony na rysunku 16.

Kod implementacji został przedstawiony w sekcji 8.6. Blok podukładu przedstawiony jest na rysunku 17.



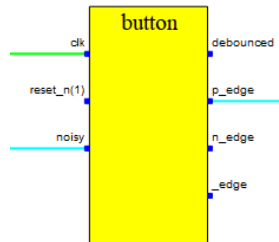
Rysunek 16: Automat skończony Meal’ego wykrywający krawędzie sygnału



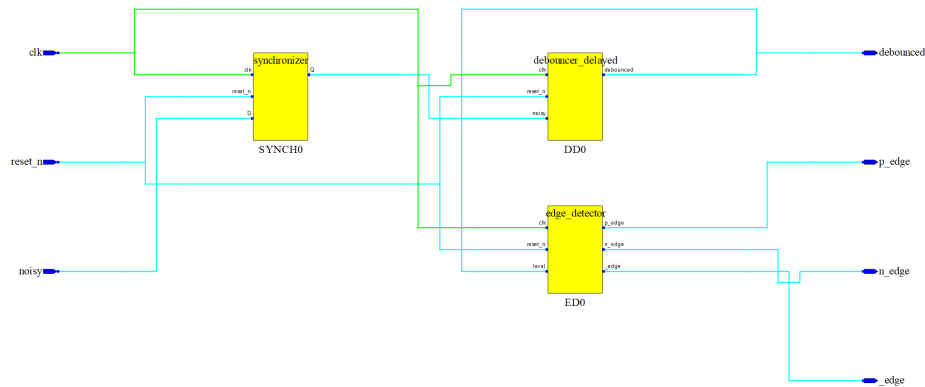
Rysunek 17: Blok podukładu detektora krawędzi

4.8 Przycisk

Dopiero po wcześniejszym zaprojektowaniu modułów: synchronizatora, filtru drgań oraz detektora krawędzi możemy stworzyć moduł przycisku. W naszym przypadku w początkowej wersji pominęliśmy synchronizację, co za skutkowało ciężkim do odnalezienia problemem kiedy to nasz program przechodził w stan niedozwolony zawieszając się i przestając reagować na wciśnięcia przycisków. Dodanie tego elementu do modułu przycisku rozwiązało ten problem. Kod implementacji został przedstawiony w sekcji 8.7. Blok podukładu przedstawiony jest na rysunku 18. Implementacja w postaci schematu widoczna jest na rysunku 19.



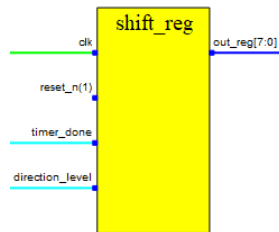
Rysunek 18: Blok podukładu przycisku



Rysunek 19: Schemat przycisku

4.9 Cykliczny rejestr przesuwający

Sposób wyświetlania pozycji naszego “węża” możemy przechowywać w rejestrze o długości ilości pól po których się porusza, gdzie inicjalnie wpisujemy pewną liczbę jedynek oznaczających obecność “węża” na tym polu oraz zer oznaczających jego brak. Najlepiej zainicjować rejestr jedynekami będącymi obok siebie (np. “11000000”), ponieważ nie chcielibyśmy go rozczłonkować na kilka segmentów. Będziemy nim poruszać (przesuwać rejestr) kiedy otrzymamy sygnał na wejściu `timer_done` oznaczającym dobiegnięcie licznika czasu do pewnej liczby. Wykorzystując multiplexer jesteśmy w stanie wybrać jeden z dwóch timerów (jeden szybszy od drugiego), a w zależności od poziomu wejścia `direction_level` będziemy przesuwali rejestr w jedną lub drugą stronę. Jako wyjście możemy bezpośrednio traktować stan rejestru, który zmapujemy na odpowiednie pola wyświetlacza. Kod implementacji został przedstawiony w sekcji 8.9. Blok podukładu przedstawiony jest na rysunku 20.



Rysunek 20: Blok podukładu cyklicznego rejestru przesuwającego

5 Załadowanie programu na płytki FPGA

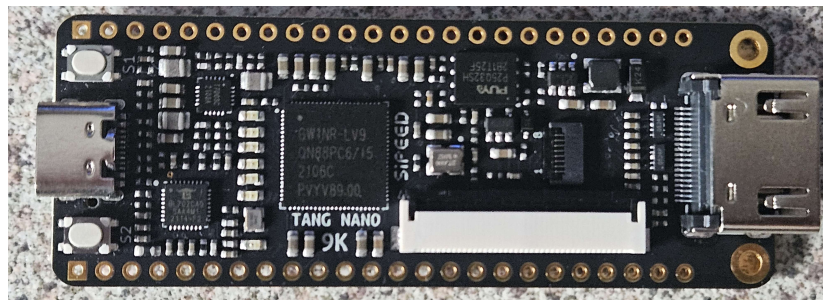
Naszą implementację przetestowaliśmy na dwóch różnych płytkach FPGA: Altera UP2 udostępnionej podczas laboratorium oraz płytce Tang Nano 9K.

5.1 Układ Altera UP2

Do stworzenia programu, konfiguracji, kompilacji oraz załadowania programu wykorzystaliśmy oprogramowanie Quartus II v9 Web Edition firmy Intel. FPGA wykorzystana na tej płytce to układ FLEX EPF10K70RC240-4. Wygląd płytki oraz efekt został pokazany w rozdziale opisującym problem 3

5.2 Układ Tang Nano 9K

Ta płytka oparta jest o mniej popularny układ GW1NR-LV9QN88PC6/I5. Nie posiada on wyświetlaczy siedmiosegmentowych, ale naszego “węża” możemy analogicznie przedstawić na 6 diodach LED zamontowanych na tej płytce (Rysunek 4). Do stworzenia programu, konfiguracji, kompilacji oraz załadowania programu wykorzystaliśmy oprogramowanie GOWIN FPGA Designer. Do zalet tej płytki należy energooszczędność, mały rozmiar oraz możliwość zasilania jak i programowania jej przez złącze USB-C. Wygląd tej płytki zamieszczono na rysunku 21.



Rysunek 21: Płytki Tang Nano 9K

6 Praktyczne zastosowania

Przykładowym zastosowaniem tego układu może być użycie go do udekorowania choinki na Święta Bożego Narodzenia. Wymagany w tym celu będzie specjalny pasek lampek, gdzie każda z nich może być sterowana osobno. Należy także zmodyfikować w niewielkim stopniu program FPGA tak, aby brał on pod uwagę większą ilość możliwych wyjść sygnału (a nie tylko 8, czyli obwód dwóch wyświetlaczy) a także aby sygnał wyjściowy był dłuższy niż dwa segmenty, dla lepszego efektu.



Rysunek 22: Przykładowe użycie układu jako sterownik do lampek choinkowych

7 Bibliografia

1. Jacek Długopolski "Scalone układy programowalne FPGA"
https://home.agh.edu.pl/~dlugopol/tc24/Lab_fpga.pdf
2. Anas Salah Eddin "ECE 3300 - Digital Circuit Design Using Verilog"
3. chipverify.com/tutorials/systemverilog

8 Kod źródłowy modułów

Zamieszczono poniżej kod w języku SystemVerilog opisujący implementację poszczególnych modułów.

8.1 snake_led.v

```
module snake_led #(parameter INIT_LED = 8'b00000011, parameter
    INIT_LED_SIZE = 8) (
    input sys_clk,                // hardware clock
    input sys_btn_1,              // hardware btn1
    input sys_btn_2,              // hardware btn2
    output wire [INIT_LED_SIZE - 1:0] led // 8 LEDS pin
);
// Button 1 with debouncer and positive edge detector
wire debounced_tick_1;
wire debounced_1;
button DEBOUNCED_BUTTON_1 (
    .clk(sys_clk),
    .reset_n('b1),
    .noisy(!sys_btn_1),
    .debounced(debounced_1),
    .p_edge(debounced_tick_1),
    .n_edge(),
    ._edge()
);
// Selected speed [0, 1]
wire speed_level;
t_ff TFF_1 (
    .clk(sys_clk),
    .reset_n('b1),
    .t(debounced_tick_1),
    .q(speed_level)
);
// Button 2 with debouncer and positive edge detector
wire debounced_tick_2;
wire debounced_2;
button DEBOUNCED_BUTTON_2 (
    .clk(sys_clk),
    .reset_n('b1),
    .noisy(!sys_btn_2),
    .debounced(debounced_2),
    .p_edge(debounced_tick_2),
    .n_edge(),
    ._edge()
);
// Selected direction [0, 1]
wire direction_level;
t_ff TFF_2 (
```

```

        .clk(sys_clk),
        .reset_n('b1),
        .t(debounced_tick_2),
        .q(direction_level)
    );
    // Fast and slow timers
    wire timer_fast_done;
    timer_parameter #(.FINAL_VALUE(1_999_999)) T_FAST(
        .clk(sys_clk),
        .reset_n('b1),
        .enable('b1),
        .done(timer_fast_done)
    );
    wire timer_slow_done;
    timer_parameter #(.FINAL_VALUE(19_999_999)) T_SLOW(
        .clk(sys_clk),
        .reset_n('b1),
        .enable('b1),
        .done(timer_slow_done)
    );
    // Rotating shift register
    reg timer_done;
    wire [INIT_LED_SIZE - 1:0] led_reg;
    shift_reg #(.INIT_REG(INIT_LED)) SHIFT_REG (
        .clk(sys_clk),
        .reset_n('b1),
        .timer_done(timer_done),
        .direction_level(direction_level),
        .out_reg(led_reg)
    );
    // Selecting timer
    always @(posedge sys_clk) begin
        if (speed_level == 'b0)
            timer_done <= timer_slow_done;
        else
            timer_done <= timer_fast_done;
    end
    // Output logic
    assign led = ~led_reg;
endmodule

```

8.2 synchronizer.v

```
module synchronizer #(parameter STAGES = 2)(
    input clk, reset_n,
    input D,
    output Q
);
    reg [STAGES - 1:0] Q_reg;
    always @(posedge clk, negedge reset_n)
    begin
        if (~reset_n)
            Q_reg <= 'b0;
        else
            Q_reg <= {D, Q_reg[STAGES - 1:1]};
        end
    assign Q = Q_reg[0];
endmodule
```

8.3 timer_parameter.v

```
module timer_parameter
  #(parameter FINAL_VALUE = 255) (
    input clk,
    input reset_n,
    input enable,
    // output [BITS - 1:0] Q,
    output done
  );
  localparam BITS = 32; // $clog2(FINAL_VALUE);
  reg [BITS - 1:0] Q_reg, Q_next;
  always @(posedge clk, negedge reset_n)
  begin
    if (~reset_n)
      Q_reg <= 'b0;
    else if (enable)
      Q_reg <= Q_next;
    else
      Q_reg <= Q_reg;
  end
  assign done = Q_reg == FINAL_VALUE;
  always @(*)
  Q_next = done ? 1'b0 : Q_reg + 1'b1;
endmodule
```

8.4 debouncer_delayed_fsm.v

```
module debouncer_delayed_fsm (
    input clk, reset_n,
    input noisy, timer_done,
    output timer_reset, debounced
);
    reg [1:0] state_reg, state_next;
    parameter s0 = 0, s1 = 1, s2 = 2, s3 = 3;
    // Sequential state register
    always @(posedge clk, negedge reset_n)
    begin
        if (~reset_n)
            state_reg <= 0;
        else
            state_reg <= state_next;
    end
    // Next state logic
    always @(*)
    begin
        state_next = state_reg;
        case (state_reg)
            s0: if (~noisy)
                    state_next = s0;
                else if (noisy)
                    state_next = s1;
            s1: if (~noisy)
                    state_next = s0;
                else if (noisy & ~timer_done)
                    state_next = s1;
                else if (noisy & timer_done)
                    state_next = s2;
            s2: if (~noisy)
                    state_next = s3;
                else if (noisy)
                    state_next = s2;
            s3: if (noisy)
                    state_next = s2;
                else if (~noisy & ~timer_done)
                    state_next = s3;
                else if (~noisy & timer_done)
                    state_next = s0;
            default: state_next = s0;
        endcase
    end
    // Output logic
    assign timer_reset = (state_reg == s0) | (state_reg == s2);
    assign debounced = (state_reg == s2) | (state_reg == s3);
endmodule
```

8.5 debouncer_delayed.v

```
module debouncer_delayed (
    input clk, reset_n,
    input noisy,
    output debounced
);
    wire timer_done, timer_reset;
    debouncer_delayed_fsm FSM0(
        .clk(clk),
        .reset_n(reset_n),
        .noisy(noisy),
        .timer_done(timer_done),
        .timer_reset(timer_reset),
        .debounced(debounced)
    );
    // X ms timer
    timer_parameter #(.FINAL_VALUE(1_999_999)) T0(
        .clk(clk),
        .reset_n(~timer_reset),
        .enable(~timer_reset),
        .done(timer_done)
    );
endmodule
```

8.6 edge_detector.v

```
module edge_detector (
    input clk, reset_n,
    input level,
    output p_edge, n_edge, _edge
);
    // Edge detector with Mealy outputs
    reg state_reg, state_next;
    parameter s0 = 1'b0, s1 = 1'b1;
    // Sequential state registers
    always @(posedge clk, negedge reset_n)
    begin
        if (~reset_n)
            state_reg <= s0;
        else
            state_reg <= state_next;
    end
    // Next state logic
    always @(*)
    begin
        case (state_reg)
            s0: if (level)
                    state_next = s1;
                else
                    state_next = s0;
            s1: if (level)
                    state_next = s1;
                else
                    state_next = s0;
            default: state_next = s0;
        endcase
    end
    // Output logic
    assign p_edge = (state_reg == s0) & level;
    assign n_edge = (state_reg == s1) & ~level;
    assign _edge = p_edge | n_edge;
endmodule
```

8.7 button.v

```
module button (  
    input clk, reset_n,  
    input noisy,  
    output debounced,  
    output p_edge, n_edge, _edge  
);  
    wire synch_noisy;  
    synchronizer #(.STAGES(2))SYNCH0(  
        .clk(clk),  
        .reset_n(reset_n),  
        .D(noisy),  
        .Q(synch_noisy)  
    );  
    debouncer_delayed DD0(  
        .clk(clk),  
        .reset_n(reset_n),  
        .noisy(synch_noisy),  
        .debounced(debounced)  
    );  
    edge_detector ED0(  
        .clk(clk),  
        .reset_n(reset_n),  
        .level(debounced),  
        .p_edge(p_edge),  
        .n_edge(n_edge),  
        ._edge(_edge)  
    );  
endmodule
```

8.8 t_ff.v

```
module t_ff (  
    input clk, reset_n,  
    input t,  
    output reg q  
);  
always @(posedge clk, negedge reset_n)  
begin  
    if (~reset_n) q <= 1'b0;  
    else if (t) q <= ~q;  
    else q <= q;  
end  
endmodule
```

8.9 shift_reg.v

```
module shift_reg #(parameter INIT_REG = 8'b00000011, parameter
    INIT_LED_SIZE = 8) (
    input clk, reset_n,
    input timer_done, direction_level,
    output [INIT_LED_SIZE - 1:0] out_reg
);
reg [INIT_LED_SIZE - 1:0] local_reg = INIT_REG;
always @(posedge clk, negedge reset_n)
begin
    if (~reset_n)
        local_reg <= INIT_REG;
    else if (timer_done)
        if (direction_level == 'b0)
            local_reg <= {local_reg[INIT_LED_SIZE - 2:0],
                local_reg[INIT_LED_SIZE - 1]};
        else
            local_reg <= {local_reg[0], local_reg[INIT_LED_SIZE - 1:1]};
    else
        local_reg <= local_reg;
end
// Output logic
assign out_reg = local_reg;
endmodule
```
